

ENEE 467: Robotics Project Laboratory

Robot Arm Kinematics

Lab 2 - Part 1 (Foundations)

Learning Objectives

The purpose of this lab is to provide a hands-on introduction to concepts in robot arm kinematics. This lab is split into two parts: In part A, we revisit concepts in forward and inverse kinematics using the Python Robotics Toolbox and the kinematic model of the UR3e. In part B, we consolidate these concepts through examples on the real UR3e robot. After completing this lab, students should be able to:

- Map joint configurations to tool poses and vice versa via forward and inverse kinematics
- Map joint and end-effector trajectories to robot motion
- Simulate unconstrained robot motion by interpolating between joint configurations
- Move a serial-chain robot (simulated using basic line plots) to trace a path in Cartesian space.

Manipulator kinematics refers to the mathematical study of how a robot arm's joint motions relate to the spatial position and orientation of its **end-effector** or **tool**. It is commonly categorized into two connected sub-problems, namely: forward kinematics (hereafter, FK) and inverse kinematics (hereafter, IK). While FK computes the tool pose from known joint variables, IK solves the more challenging problem of finding joint configurations that achieve a desired tool pose. Together, they form the backbone of higher-level motion planning and control, and are thus crucial for robotic manipulation. Therefore, this lab is designed to equip you with the tools and mathematical thinking to understand kinematic modeling, practice kinematics-based problem solving, and prepare for ROS 2 motion planning and control in later labs.

Related Reading

1. K. Lynch and F. Park, *Modern Robotics: Mechanics, Planning, and Control*, Cambridge University Press, 2nd ed., 2019 (§4, §5, and §6).
2. ROS URDF Tutorials. [External Link](#).

Lab Instructions

Follow the lab procedure closely. Some code blocks have been populated for you and some you will have to fill in yourself. Pay close attention to the lab [Questions](#) (highlighted with a box). Answer these as you proceed through the lab and include your answers in your lab writeup. There are several lab [Exercises](#) at the end of the lab manual. Complete the exercises during the lab and include your solutions in your lab writeup.

List of Questions

Question 1.	4
Question 2.	6

Question 3.	7
Question 4.	8
Question 5.	9
Question 6.	11
Question 7.	12
Question 8.	13

List of Exercises

Exercise 1. Tracking a Cartesian Square with IK (20 pts)	16
Exercise 2. Numerical Inverse Kinematics Solver (30 pts)	17

How to Read This Manual

Throughout the manual, boxes beginning with a **Question #** title contain questions to be answered as you work through the lab:

Question 0. A question that you should answer as you work through the lab manual.

Boxes with a  symbol (like the one below) are extra theoretical details provided for your enlightenment or just general information about the lab. We recommend that you read them during the lab, but they may be skipped during the lab and revisited at a later time (e.g., while working on the lab report):

 Some extra context and info.

Terminal commands and code blocks will be set in gray-colored boxes (usually for Python syntax)

```
import math # some important comment you should totally read
```

while terminal outputs will be set in yellow-colored boxes

```
some terminal output
```

Files and directories will be set in magenta color with a teletype font, e.g., `~/some/dir/to/a/file.txt`, code variable names and in-text Python commands or snippets will be set in blue teletype font, e.g., `a = [0, 0, 0]`, while Python type hints (and occasionally XML tags) will be set in green teletype font, e.g., `save(path: str)` signifies that the argument to the `save` function is a string. Lastly, figure captions go below figures, table captions appear above tables, and [cyan text](#) highlights a link to an external online reference.

1. Lab Procedure

1.1 Prerequisites

This lab assumes some familiarity with the Bash shell at the level of file system navigation and basic read-write operations via the Command Line Interface. We also assume that you have done some prior Python programming, and optionally, have some exposure to numerical computation with the NumPy Python library, basic linear algebra, and rigid-body motion.

1.2 Getting this Lab's Code

To begin, in a new terminal session, navigate to the `~/Labs` directory and then clone the lab repo:

```
cd ~/Labs
git clone https://github.com/ENEE467-F2026/lab-2.git
cd lab-2/docker
```

Verify that the lab-2 image is built on your system by running the following command

```
docker image ls
```

Make sure that you see the lab-2 image in the output (the tag, container size, and time information will differ from those on your machine):

```
lab-2-image    latest      88f2baf1996e    2 minutes ago    6.01GB
```

Now launch the Docker container with the following command

```
userid=$(id -u) groupid=$(id -g) docker compose -f lab-2-compose.yml run \
--rm lab-2-docker
```

You should be greeted with the following prompt indicating you are now inside the Docker container:

```
(lab-2) robot@docker-desktop:~$
```

Test your setup by executing a Python script within the `lab-2/src` directory in the terminal running the container:

```
cd ~/lab-2/src
python 2r_dh.py
```

The script should print a table showing the DH parameters of a simple 2R manipulator. You may now proceed with the rest of the lab procedure.

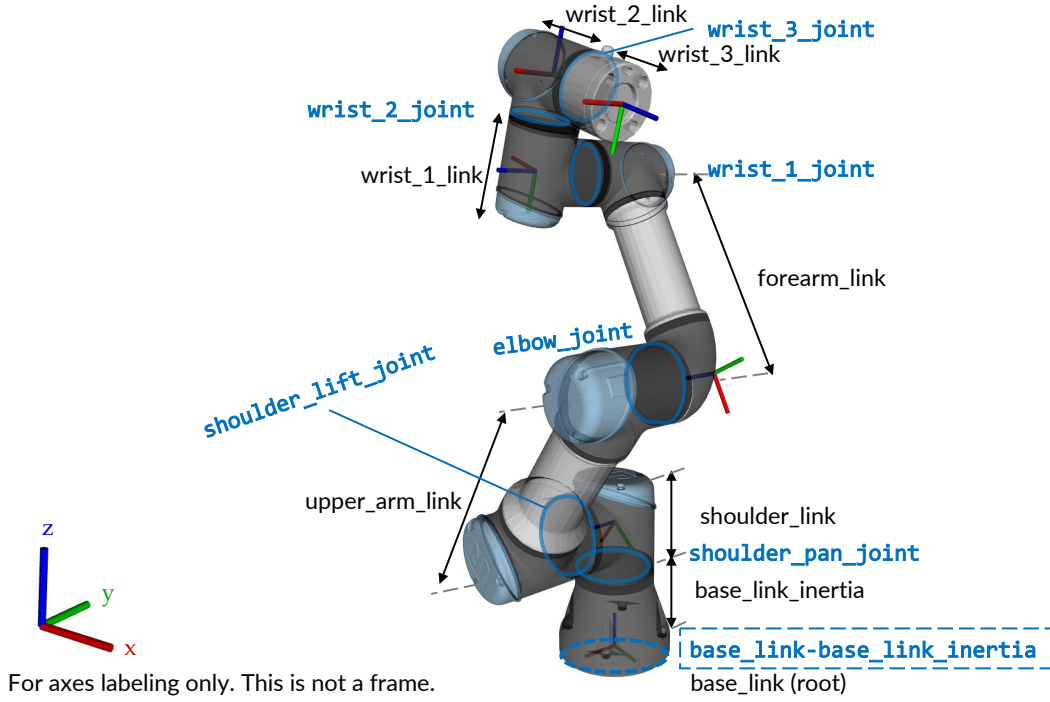


Figure 1: Link frames and joints of the UR3 robot. Joint names are set in blue color, while link names are set in black. The UR3's end-effector frame is typically taken to be the attached to the `tool0` link (not annotated), a fictitious link whose origin differs from that of the `wrist_3_link` frame only by a rotation, i.e., the translation component of the homogeneous transformation between both frames is the zero vector in $\mathbb{R}^3$. The `base_link-base_link_inertia` joint is fixed.

2. Forward Kinematics of Serial Chains

2.1 Computing FK from Homogeneous Transformation Matrices

Because robotic tasks are most naturally expressed in the tool frame but we can typically only measure the **position** of the **joints** via encoders and have no way of ascertaining the end-effector's position without external instrumentation, we need a mapping between the joint angles we can measure and the end-effector's pose. The FK equations (Equation (1)) provide this very important mapping, typically in terms of the manipulator's link lengths and the joint configuration (q):

$$f(q) = \begin{bmatrix} R(q) & p(q) \\ 0 & 1 \end{bmatrix} \in SE(3). \quad (1)$$

To compute $f(q)$ for a kinematic chain, we compose the homogeneous transformations between consecutive links (from the root link to the end-effector) as

$$f(q) = T_{\text{link1}}^{\text{root}} \cdot T_{\text{link2}}^{\text{link1}} \cdot \dots \cdot T_{\text{ee_link}}^{\text{linkN}}, \quad \text{where each } T \text{ term is an } SE(3) \text{ object.} \quad (2)$$

Work through [Question 1](#) to practice computing a robot's FK from its URDF.

Question 1.

- (a). Within the same file you modified in ??, modify the block highlighted with a `TODO` comment to find the **matrix product** of **all** transformation matrices in ?? (a) in ascending order of the frame pair's index within the kinematic chain, i.e., $T_{\text{shoulder_link}}^{\text{base_link}} \cdot T_{\text{upper_arm_link}}^{\text{shoulder_link}} \cdot \dots \cdot T_{\text{tool0}}^{\text{wrist_3_link}}$. Include the final matrix product in your report.

Note: To multiply two `spatialmath.SE3` objects, use the `*` operator; if working instead with `spatialmath.SE3` objects converted to NumPy arrays (e.g., `spatialmath.SE3.A`), then you can use the `@` operator or the `numpy.matmul` function:

```
import numpy as np
A = np.eye(4)
B = A
mat_prod = np.matmul(A, B) # works only if A and B are NumPy arrays
```

- (b). Compare your answer in Question 1 (a) with the homogeneous transformation matrix between the link pair: (`base_link`, `tool0`) (use the same function in (a) to find this). In your report, provide the **modified code block** (the raw text or a screenshot is fine) as well as reasoning (in no more than **two** sentences) why you think the matrices are the same or different (depending on your results).
- (c). Modify the variable `q` in the file `q5c.py` to `[1.5, -2.5, 1.5, -1.05, -1.55, 0.0]`. Then change the `end` argument of the `robot.fkine` method to `'tool0'`. Run the modified script. The script will print an SE(3) matrix to the terminal. Read off the translation component of the matrix, and provide the translation vector in your report. Compare the printed SE(3) matrix and your result in (b). Verify that they are the same.

As you've seen from the questions up to this point, for a two or three link manipulator, it is a simple matter to compute FK by hand by straightforward matrix multiplication; anything more complex, and the calculations quickly become tedious. And because FK changes with the joint configuration, we don't want to keep multiplying matrices for each configuration anytime we want to do FK. Fortunately, we can easily compute FK for any serial chain using its kinematic model (contained in the URDF spec or specified as DH parameters) using a handy method provided by RTB4PY. We go over an example and question in the next section.

2.2 Computing FK for a Kinematic Chain with the Toolbox

Suppose we load a standard UR3 model and specify an array of joint angles (we will use variables namespaced by `q` for Python objects related to joint configurations). We can compute the resulting end-effector pose (as a transformation matrix) using the method `fkine` from the `UR3` class instance. Inspect the `src/ur3_fk.py` file for example Python code (shown below) that implements FK:

```
import roboticstoolbox as rtb
import numpy as np # library containing important kernels for matrix math

robot = rtb.models.DH.UR3() # Load a UR3 model
q = np.array([0, -np.pi/4, np.pi/4, 0, np.pi/3, 0]) # joint configuration
```

```
T = robot.fkine(q) # Compute FK
```

Using the `fkine` method, the script computes a 4×4 homogeneous transformation matrix $\mathbf{T}$, which encodes our robot's end-effector position and orientation in task space (for the UR3, this is the **tool0** frame). Issue a `print(T)` statement on a new line at the end of the script and run it. You should see the following matrix output in the terminal

```
0.5      0      -0.866   -0.4565
-0.866    0      -0.5     -0.1533
0         1       0       0.2388
0         0       0        1
```

In the homogeneous transformation matrix ($\mathbf{T}$), as you learned in class, the first three rows of the last column correspond to the end-effector's position, while the rotation matrix (the 3×3 matrix block in the first three columns of $\mathbf{T}$) defines the end-effector's $SO(3)$ orientation. We can plot the robot at this configuration to confirm visually that the FK calculations are correct. After the `print(T)` line in the `ur3_fk.py` script you just modified, add the following new line

```
robot.plot(q, block=True, jointaxes=True, shadow=True, name=True)
```

You should see a Matplotlib window showing a simple line plot of the robot.

Question 2. By modifying the `ur3_fk.py` script, try computing FK at a configuration near the robot's upper limit (accessible using `robot.qlim[1]`, where `robot.qlim` is a 2×6 NumPy array). What is the **position** of the robot's end-effector? Plot the robot at the upper limit configuration using the `robot.plot` command. Notice how some links appear folded over one another, creating an undesirable configuration. Save the Matplotlib figure and add it to your report, along with the array representing the robot's end-effector position.

Great, you just computed FK! But does that mean the robot can actually reach the configuration $\mathbf{q}$? FK only maps joint angles to an end-effector pose; it says nothing about the quality of that configuration. In fact, the math may work out even if the joint angles place the robot at or near a **singularity**, where the end effector loses motion in one or more directions.

3. Kinematic Singularity & The Manipulator Jacobian

3.1 The Manipulator Jacobian

To move from the FK realm of tool **positions** and **orientations** to **velocities**, we need a new object — the **Jacobian matrix** or **Jacobian** for short. The Jacobian is an important object in kinematics you'll see repeatedly. Mathematically, the (**space**) Jacobian J relates the velocity of the end-effector in the **base (space) frame** ($\dot{x}$) to joint-space velocities ($\dot{q}$), and depends on the configuration, q :

$$\dot{x} = J(q)\dot{q}, \text{ with } J = \begin{bmatrix} J_v(q) \\ J_\omega(q) \end{bmatrix}. \quad (3)$$

J is a block matrix with 6 rows and n columns (corresponding to the number of joints) and is typically decomposed into **linear** (top 3 rows) and **angular** (bottom 3 rows) velocity components. For example, the Jacobian corresponding to the UR3 at the configuration $[0, -\frac{\pi}{2}, 0, -\frac{\pi}{2}, 0, 0]^T$ is

[[0.19 -0.54 -0.3 -0.09 0.08 0.] # }	lin. vel. along base x
[0. 0. 0. 0. 0. 0.] # J_v	lin. vel. along base y
[0. 0. 0. 0. 0. 0.] # }	lin. vel. along base z
[0. 0. 0. 0. 0. 0.] # }	ang. vel. about base x
[0. -1. -1. -1. 0. -1.] # J_omega	ang. vel. about base y
[1. 0. 0. 0. 1. 0.] # }	ang. vel. about base z

which makes sense, since just by inspecting the first row, we see that motion in all joints except the last joint yields instantaneous velocity along the base x axis, and the last column is all zeros (except for rotation about the y axis) since the last joint can only **rotate** the tool frame but not **translate** it. Since the FK equations can be applied to any frame pair in a kinematic chain, we have all the components we need to compute J , so let's write a simple Jacobian calculator to concretize things.



It helps to picture the Jacobian as a **summary of joint velocity contributions**:

Revolute joints contribute a column $\in \mathbb{R}^6$:

Prismatic joints contribute:

$$J_i = \begin{bmatrix} z_{i-1} \times (p_{ee} - p_{i-1}) \\ z_{i-1} \end{bmatrix} \quad \begin{matrix} J_{v_i} \\ J_{\omega_i} \end{matrix} \quad (4)$$

$$J_i = \begin{bmatrix} z_{i-1} \\ 0 \end{bmatrix} \quad \begin{matrix} J_{v_i} \\ J_{\omega_i} \end{matrix} \quad (5)$$

where p_{ee} is the end-effector position (translation of T_n^0), p_{i-1} is the position of link $i-1$ (translation of T_{i-1}^0), z_{i-1} is the unit vector along joint i 's motion axis ($0 \in \mathbb{R}^3$ for prismatic joints), and $\times$ denotes the cross product. Joint i connects link $i-1$ (child) to link i (parent), with the base as link 0; the origin of joint i in the base frame is the end of its child link, or the origin if at the root. With each J_i , the full Jacobian is $J = [J_1 \mid J_2 \mid \dots \mid J_n]$, where $\mid$ denotes column-wise concatenation.

3.2 Writing a Simple Jacobian Calculator

As you can probably tell from Equations (4) and (5), computing $J(q)$ amounts to simply stacking columns into matrices, one for each joint of the kinematic chain. We can perform this “stacking” operation for each Jacobian component, as you’ll explore in the next question.

Question 3. Open the file `compute_urdf_jac.py`. The file contains a partial implementation of a simple Jacobian calculator function (`compute_jacobian`) that computes the linear and angular velocity Jacobian components using forward kinematics kernels from a custom URDF parser class we have provided. The script populates a NumPy array of zeros with the computed values, and returns the final matrix as the Jacobian.

- By modifying the `TODO`-highlighted blocks, compute the Jacobian by implementing Equation (4) for the **revolute joint** (you can use NumPy’s `cross` method to compute the cross product). For example, to compute the Jacobian for the case of a prismatic joint (i.e., Equation (5)), you would set:

```
Jv = z_i_prev
Jw = np.zeros(3)
```

for each joint in the kinematic chain. In your report, provide the resulting Jacobian, **as well as** (text or image) code snippets highlighting the **modified** lines that produced your result.

- (b). After the [Simple URDF Jacobian Calculator](#): print statement within the [compute_urdf_jac.py](#) file, add the following lines:

```
robot_rtb = rtb.models.UR3()
print("-----")
print(f"Toolbox Jacobian Result: \n{np.round(robot_rtb.jacob0(q_arr,\nend='tool0'), 4)}")
```

Note: Ensure the print statements are properly indented after pasting. Run the code and compare the matrix printed after **Toolbox Jacobian** with your result from (a), verifying agreement to three decimal places. Include the final [compute_urdf_jac.py](#) file with your report.

3.3 Computing the Jacobian using the Toolbox

In RTB4PY, we can directly compute the Jacobian using yet another one-liner, **robot.jacob0(q)**, a function you've already seen from [Question 3](#) (b). The **robot.jacob0** method yields the robot's $6 \times n$ Jacobian at the joint configuration **q**, expressed in the base (**0**) frame. To use it, simply call the method from the corresponding `rtb.models.DH.<RobotClass>` or `rtb.models.<RobotClass>` robot model constructor instance. An example using the UR3 DH model follows:

```
robot = rtb.models.DH.UR3()
J = robot.jacob0(q) # assuming q is not NoneType and len(q) == 6
```

Question 4. Using the toolbox, compute the Jacobian for the UR3 at the configuration:

```
q = [-1.860, -2.990, 0.670, -2.000, 1.610, -0.007]
```

What are the linear (J_v) and angular velocity (J_ω) Jacobians? Include these in your report.

🍃 For a 6-DoF chain ($n = 6$), the Jacobian $J(q)$ is square and invertible if $\det(J) \neq 0$. For non-square cases, we use the **Moore–Penrose** pseudo-inverse $J^\dagger$ (e.g., `numpy.linalg.pinv(J)`). RTB4PY also provides related functions such as `robot.jacob0_dot` and `robot.jacobe`, though `jacob0` is sufficient here.

3.4 Identifying Singular Manipulator Configurations

Once the Jacobian is computed, determining whether the robot is at a singular configuration is simply a matter of computing the **Jacobian's inverse**. From linear algebra, we know that a non-invertible matrix is **rank-deficient**, meaning one or more rows or columns can be written as

non-zero factors of another. So if $\mathbf{J}(\mathbf{q})$ is an $n \times n$ matrix and its rank (or equivalently, in most cases, the existence of its inverse) is less than n , then $\mathbf{q}$ must be singular. Thus, we can check singularity by computing the inverse of `robot.jacob0(q)`. To compute the inverse or rank of a matrix, NumPy provides handy functions:

```
import numpy as np
J = robot.jacob0(q) # assuming q and robot = rtb.DH.UR() are not NoneType
J_inv = np.linalg.inv(J) # inverse
rank_J = np.linalg.matrix_rank(J) # rank
```

Let's try an example. Open the file named `ur3_jac.py`, containing the following code (truncated):

```
q_regular = [-1.860, -2.990, 0.670, -2.000, 1.610, -0.007] # Non-singular q
q_at_lim = robot.qlim[0] # lower joint limit of robot, theta = -\pi
# Jacobians
J_reg = robot.jacob0(q=q_regular)
J_at_lim = robot.jacob0(q=q_at_lim)

# Matrix ranks
rank_J_reg = np.linalg.matrix_rank(J_reg)
```

Two manipulator configurations are included in the script: a non-singular configuration, `q_regular` and a singular configuration at the lower joint limit ($-\pi$ for all joints), `q_at_lim`. The script computes the Jacobians corresponding to each configuration and the associated matrix ranks, and then checks whether the inverse of the Jacobians exist using a nuanced approach involving the **condition number** and not the inverse directly (see: [here](#)). Run the script, and verify that the following output is printed to the terminal:

```
The rank of J_reg is 6, so q_reg is non-singular,
    and it is true that its inverse exists
```

```
The rank of J_at_lim is 5, so q_at_lim is singular,
    and it is not true that its inverse exists
```

Question 5. By modifying the `q_to_test` and `J_to_test` variables, along with attendant `print` statements in the `ur3_jac.py` file (all relevant code blocks are highlighted with a `TODO`), determine whether or not the following configurations are singular by comparing **both** the Jacobian inverse (if it exists) and the value returned by the provided `check_inv` function. Report your answer as **Singular** or **Non-singular** in each case (**Note**: you only need to report one answer per configuration. Use your best judgment to determine which method to trust: the de facto and intuitive inverse, or the lesser-known condition number approach). Each array is a value you should set `q_to_test` to:

- (i). $[-1.360 \quad -1.990 \quad 0.570 \quad -1.000 \quad 1.510 \quad -0.002]^T$
- (ii). $[3.1415 \quad 3.1415 \quad 3.1415 \quad 3.1415 \quad 3.1415 \quad 3.1415]^T$
- (iii). $[-0.380 \quad -1.330 \quad 0.420 \quad -0.000 \quad 0.510 \quad -0.054]^T$

$$\text{(iv). } [0 \ 3.14159265 \ 3.14159265 \ 3.14159265 \ 3.14159265 \ 0]^T$$

$$\text{(v). } [1.65762426 \ 3.14159265 \ 3.14159265 \ 3.14159265 \ 3.14159265 \ 1.65762426]^T$$

4. From Single Joint Configurations to Trajectories

To move from configuration to motion, we must define a new object that specifies a *sequence* of configurations to step through over **time** to achieve the required motion. This object is the **joint trajectory**, typically denoted by $q(t)$, with t denoting the time variable.

4.1 Mapping Joint Trajectories to Robot Motion

While it might be straightforward to define a trajectory comprising a few joint configurations by hand, especially if the end-points are trivially close, defining a trajectory for say 50 steps manually is not only unreasonable but is also guaranteed to be fraught with errors. Luckily, we have a single-call method to simplify this process within the toolbox, namely the **jtraj** method.

The **jtraj** method takes three arguments: the start configuration, the end configuration, and the desired number of intermediate steps between both configurations, and produces a joint-space trajectory comprising as many configurations as the specified number of steps. Open the [src/ur3_jsp_motion.py](#) file which should contain the following lines of code (truncated):

```
robot = rtb.models.DH.UR3() # Load a UR3 model

# Define the start and end configurations of the robot arm
q_start = [0.0, -np.pi/2, 0.0, -np.pi/2, 0.0, 0.0]
q_end = [-1.860, -2.990, 0.670, -2.000, 1.610, -0.007]

traj = rtb.jtraj(q_start, q_end, 50) # Interpolate trajectory (50 steps)
robot.plot(traj.q, block=True)      # Animate the motion
```

Run the script. A Matplotlib window should appear showing an animation where the robot moves from an initial upright configuration to a final configuration. In the file, the trajectory created by **traj** is a multidimensional NumPy array containing **50** joint configurations between **q_start** and **q_end**, inclusive. Confirm this by checking the trajectory's length. Comment out the **robot.plot()** call to disable the animation, and add the following two lines at the end of the Python file

```
print(f"The trajectory's shape is: {np.shape(traj.q)}")
print(f"The trajectory contains {len(traj.q)} configurations \n \
      each of length {np.shape(traj.q)[1]}")
```

Run the script. It should print:

```
The trajectory's shape is: (50, 6)
The trajectory contains 50 configurations
      each of length 6
```

verifying that there are indeed 50 joint configurations in the trajectory, with each trajectory containing 6 joint angles, one for each joint of the UR3.

Question 6. Run [ur3_jsp_motion.py](#) file for 30, 60, 100, 120, and 150 steps (joint configurations). Log the approximate execution time (printed at the top-right corner of the Matplotlib window) for each run, i.e., the time (in seconds) it roughly takes for the robot to reach **q_end**. Include a plot of number of steps vs. execution time in your report.

4.2 Computing Singularity-Free Joint Trajectories

We can now put the Jacobian math from [Section 3](#) to some more use. Whereas the example in [Section 3.4](#) determined if a **single** configuration was singular using the Jacobian, in practice, we'd like to plan **entire** robot arm motions in such a way that singular configurations are avoided. That is, we want to ensure that each configuration in the computed trajectory is non-singular. We have all the tools to do this. Open the file `ur3_jsp_jac.py`. You'll find that this file is somewhat similar to the `ur3_jsp_motion.py` script except that now instead of an initial and a final configuration, we have included an intermediate configuration by stacking trajectory segments, setting this middle configuration to a known singular configuration on purpose.

```
q_start = [0.0, -np.pi/2, 0.0, -np.pi/2, 0.0, 0.0]
q_at_lim = robot.qlim[0] # singular intermediate config
q_end = [-1.860, -2.990, 0.670, -2.000, 1.610, -0.007]
```

The script implements two versions of the trajectory planner we saw in the `ur3_jsp_motion.py` example: a **naive** planner, where *all* configurations are included in the planned trajectory, and a **singularity-aware** planner that *filters* the trajectory from the naive planner by removing configurations near or at kinematic singularities using the method we saw in [Section 3.4](#). The code block that implements this filtering routine is:

```
def avoid_singular_q(traj):
    traj_non_sing = np.empty_like(traj[0,:])
    traj_sing = np.empty_like(traj[0,:])
    for q in traj:
        if check_inv(robot.jacob0(q)): # see code for definition
            traj_non_sing = np.vstack([traj_non_sing, q])
        else:
            traj_sing = np.vstack([traj_sing, q])
    return traj_non_sing, traj_sing
```

Run the code. You should see two animations play sequentially. In the first animation, the robot visits every configuration in the trajectory, including the singular ones, 'happily' folding on top of itself (yes, as you will notice from the animation and [Question 7](#), including a singular configuration smack in the middle of configurations to visit creates even more singular configurations). In the second animation, however, the robot visits fewer configurations, and is actively steered away from reaching the singular configurations as the trajectory progresses. The `VERY_HIGH_NUM` variable in the `check_inv` function controls the filter's strength. At `VERY_HIGH_NUM = 1000`, for instance, 10 singular configurations are expunged from the final trajectory, leaving 40 valid ones; at 100, only 24 make it through.

Question 7. By modifying the `ur3_jsp_jac.py` file, test the performance of the singularity-avoiding planner with the variable `VERY_HIGH_NUM` on the `TODD`-highlighted line set to the following values: `1e3`, `1e4`, `1e5`, and `1e6`. Report the number of valid (non-singular) trajectories in each case. Hint: You should notice an upward trend.

5. Inverse Kinematics

5.1 Unpacking IK

IK is simply the process of finding the joint configuration q that achieves a **desired** end-effector pose $T_d \in SE(3)$. Unlike FK, which is mostly straightforward ($T = f(q)$), IK requires solving the nonlinear inverse mapping $q = f^{-1}(T_d)$ using one of two primary approaches: **analytical** methods, which are often difficult to derive, and **numerical** methods, which are more tractable but approximate. In Exercise 3, you will solidify your understanding of inverse kinematics by completing a basic numerical IK solver. For now, let's see what IK methods the toolbox provides.

5.2 Computing IK with the Toolbox

To compute IK via the toolbox, we can call one of several algorithm-specific methods from the same `robot.models.DH.UR3` class instance we've used in previous sections. The methods are summarized on Table 1. Open the file `ur3_ik.py` for an example that implements IK (a truncated version is included below). In the example, we want the end-effector to reach a desired pose expressed as a desired position or translation (using the method `sm.SE3(x, y, z)`) and a desired 90° counterclockwise rotation about the Z axis (using the method `sm.SE3.Rz(angle)`). We apply IK here to find the joint configuration(s) that achieve this pose.

```
import roboticstoolbox as rtb
import numpy as np # library containing important kernels for matrix math
import spatialmath as sm # library containing important kernels for tf math

robot = rtb.models.DH.UR3() # Load a UR3 model; creates a UR3 class instance
T_goal = sm.SE3(0.5, 0.2, 0.3) * sm.SE3.Rz(np.pi/2) # Desired EE pose

# Compute IK
solution = robot.ikine_LM(T_goal) # Levenberg-Marquardt solver
print(solution.q) # Joint angles (if solution found)
```

Run the code. You should see a 6D array of joint angles in radians, which corresponds to the solution found by the IK solver:

$$\begin{bmatrix} -2.97125889 & 0.0532965 & -0.43369849 & -1.19039433 & 1.57079633 & -0.17033376 \end{bmatrix}^T.$$

Note that your solution might be different from the previous array, but the solver will return a solution regardless.

📌 Intuitively, we can think of IK as essentially “nudging” the joints in the direction that reduces the tool pose error ($x_d - f(q_k)$), converging to a solution over time. If no convergence occurs within the preset maximum number of iteration steps or a collision occurs, the procedure ends and reports failure. In addition, IK may yield multiple solutions or none at all, depending on the desired pose and joint limits, as numerical IK solvers often include joint limits as constraints. To ensure repeatability, it may thus be necessary to “freeze” or store desirable trajectories, especially for tasks involving repeated motions.

Table 1: Common numerical methods for IK. The string in each function’s name identifies the numerical optimization IK solver associated with that function.

Function	Algorithm	External Reference
ikine_LM	Levenberg–Marquardt	Wikipedia
ikine_GN	Gauss–Newton	Wikipedia
ikine_NR	Newton–Raphson	Wikipedia
ikine_QP	Quadratic Programming	Wikipedia

Question 8.

- Run the `ur3_ik.py` script THREE times, freezing the solver to the **LM** and **GN** types for each triple run. Collect the different joint configuration solutions (rounded to 4 decimal places) each solver type yields each time in a table and include the table in your report.
- Using the same IK script but only with the LM solver, find the joint configuration corresponding to the tool pose at the robot’s lower and upper joint limits, i.e., the two elements of `robot.qlim`. Note: one IK solution is enough for each joint limit. Include the values (rounded to 4 d.p.) in your report.
- For the two solutions you obtained in (b), compute the magnitude of the difference between the corresponding configuration at the joint limits and the IK solution. We provide an example for the lower limit below (assuming the IK solution corresponding to this limit is stored in the variable `sol_l`), but you should also compute the corresponding quantity for the upper limit as well:

```
np.linalg.norm(robot.qlim[0] - sol_l.q)
```

In your report, explain in **one** sentence why you think your answer is (or isn’t) non-zero.

5.3 Mapping End-Effector Trajectories to Robot Motion via IK

You just crunched numbers to compute IK! Let’s now explore a few things we can do with this new tool. We’ll use a tracking example like in ??, but with a circular trajectory. Open the file `ur3_ik_circle.py`. The relevant blocks are highlighted below.

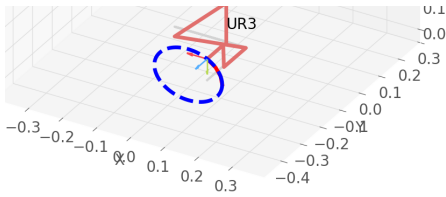
```
# Create a circle in task space (ZX plane; y constant)
r = RADIUS # m
t = np.linspace(0, 2*np.pi, NUM_PTS_TSP_CIRC) # controls smoothness; see code
x = r * np.cos(t)
y = ee_pose_des_t[1] * np.ones_like(x)
z = r * np.sin(t)
tsp_circle = np.vstack((x, y, z)).T

# Sample points on the task-space circle
indices = np.linspace(0, NUM_PTS_TSP_CIRC - 1, NUM_SAMP_PTS, dtype=int)
sampled_pts = tsp_circle[indices]

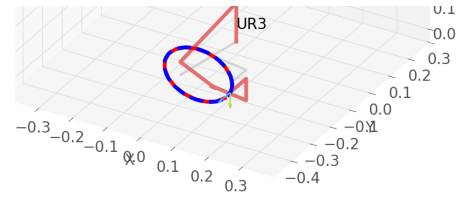
# Compute IK for each sampled point and add to trajectory
traj = np.empty((0, 6))
for sampled_pt in sampled_pts:
```

```
T_goal = sm.SE3(np.array(sampled_pt)) * sm.SE3(sm.S03((ee_pose_des_R)))
q_sol = robot.ikine_LM(T_goal).q
traj = np.vstack([traj, q_sol])
```

The main logic of the script is simple: we define a circle in task space. Since there are an infinite number of points on any geometric circle, we sample a few, making sure to cover the entire perimeter. We then compute IK for each sampled point and add the solution to our robot's trajectory. Try running the script. You should see an animation of the UR3 tracking a task-space circle using IK-computed joint configurations. However, while the task space trajectory appears smooth, the joint space trajectory appears jerky and fragmented. We will revisit this jerky motion in a later lab.



Initial configuration



Final configuration

Figure 2: UR3 tracking a task-space circle using numerical inverse kinematics.

6. Exercises

Exercise 1. Tracking a Cartesian Square with IK (20 pts)

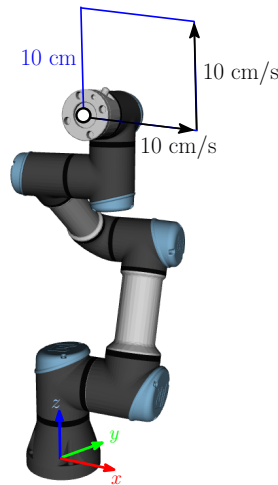


Figure 3: UR3 and target Cartesian square path. The white dot marker on the robot's last link highlights a waypoint while arrows signify the motion direction.

Robots can move their end-effectors using different strategies. Velocity kinematics integrates desired end-effector velocities to produce smooth motion but does not guarantee **precise** positions. However, many industrial manipulation tasks, such as assembly, pick-and-place, or welding, require inverse kinematics (IK) to compute the joint configuration needed to reach **exact** end-effector poses.

In this exercise, you will complete the square-drawing task for the UR3 using only IK by modifying the script `exercise.py` in the `src/exercise` folder. To accomplish this, you need to:

1. Specify the **corner points of the square** (waypoints) in Cartesian space (see [Figure 3](#))
2. Solve the **IK problem** for each corner using a Levenberg–Marquardt solver to compute joint configurations.

The relevant blocks within the code are highlighted with a `TODO` comment, and the deliverables for this exercise are enumerated below:

Deliverables:

- The 6×6 IK solution array that is printed to the terminal when your code runs
- The modified `exercise.py` file that produced the IK solution results (see the file-naming convention in the next section ([Section 7](#))).

Exercise 2. Numerical Inverse Kinematics Solver (30 pts)

Numerical IK methods such as the Jacobian pseudo-inverse method and the Damped Least Squares algorithm use the **pseudo-inverse** of the manipulator Jacobian $J^\dagger$ (here's that object again) to update the joint configuration iteratively until **convergence to a solution** (i.e., to a **joint configuration**) is achieved, as in

$$q_{k+1} = q_k + \gamma_k J^\dagger(q_k) (x_d - f(q_k)), \quad (6)$$

where $f(q_k)$ is the end-effector pose at time step k realized from computing FK, and γ_k is a step size parameter that may be constant or vary with k . Convergence is achieved if

$$\|q_{k+1} - q_k\| \leq \epsilon, \text{ for } k < \infty, \quad (7)$$

and where $\epsilon > 0$ is a convergence tolerance. **Note:** γ_k and ϵ are not the same quantity.

To implement our IK solver, we need a few components:

- i. a convergence tolerance, ϵ to implement Equation (7)
 - ii. a Jacobian-computing kernel that outputs Jacobians given a joint configuration, making sure that the configurations are non-singular and within joint limits before computing anything
 - iii. An FK map to compute the task-space error in Equation (6), i.e., $x_d - f(q_k)$
 - iv. A step-size schedule for γ_k . For simplicity, we'll set γ_k to a constant ($\gamma \in (0, 1]$) for all k .
- Fortunately, the necessary components have already been developed in earlier sections and questions; they only need to be integrated into a single function. Your task is to finish the partial implementation of the numerical inverse-kinematics solver located in `src/min_ik_solver.py`. The deliverables for this exercise are:

Deliverables:

- (a). In the file `min_ik_solver.py` is a partial implementation of a minimal IK solver (`compute_ik`) that takes as argument the desired $SE(3)$ pose (`x_d`), a positive convergence tolerance (`eps`), an initial configuration guess (`init_guess`), a value for γ (stored in the variable `gamma`), and a specification of the maximum number of iterations (`max_iters`) after which the routine reports failure if a solution is not found within the tolerance specified by `eps`. The function should output a set of joint angles in radians whose FK work out to be the desired $SE(3)$ pose argument. By modifying the `TODO`-highlighted blocks, **complete** the numerical IK routine by implementing Equation (7) and Equation (6), respectively. Your function should return three results upon convergence:
 - The final configuration, stored in the variable: `config_k`
 - The norm of the task-space error, stored in the variable: `error`
 - The number of iterations at convergence.

Using the provided `main` function, test your IK function with the following desired $SE(3)$ Cartesian poses:

- **Test Case 1:** $T_1 = T_x(0.3)T_y(0.2)T_z(0.35) R_x(0)R_y(0)R_z(0)$
- **Test Case 2:** $T_2 = T_x(0.4)T_y(-0.1)T_z(0.3) R_x(0)R_y(\pi/4)R_z(0)$
- **Test Case 3:** $T_3 = T_x(0.2)T_y(0.1)T_z(0.4) R_x(\pi/2)R_y(0)R_z(\pi/2),$

noting down the final configuration, task space error norm, and number of iterations at convergence (these metrics will appear in the terminal if your programming logic is correct). In your report, provide the joint configuration, number of iterations, and error for all three test cases above.

- (b). Compare your solutions in (a) with the solutions returned by the following code block (add it to the `main` function in `min_ik_solver.py` after the `print(f"IK Solution ...")` line, and run the script):

```
# RTB4PY for verification
print(f"IK Solution (RTB): {np.round(ik_sol_rtb.q, 3)}, \
      num_iterations: {ik_sol_rtb.iterations}, \
      error: {np.round(ik_sol_rtb.residual, 5)}\n")
```

Explain in at most **two** sentences in your report why you think the results in (a) and (b) are different.

- (c). By computing forward kinematics for the solution returned by your solver in (a), verify that the desired Cartesian poses are recovered for each test case. In your report, provide the resulting 4×4 homogeneous transformation matrix in each case, along with a text snippet of the code that produced your FK results.
- (d). Your modified Python script (include this as an attachment with your Canvas submission).

7. Conclusion and Lab Report Instructions

After completing the lab, summarize the procedure and results into a well written lab report. Include your solutions to all of the question boxes in the lab report. Refer to the [List of Questions](#) to ensure you don't miss any. Include your full solution to the exercises in your lab report. If you wrote any Python programs during the exercises, include these files as a separate attachment, appending your team's number and last names (in alphabetical order) to the file, e.g., **Jane Doe, Alice Smith, and Richard Roe** in **Team 90** submitting their modified `main.py` would submit the file `main_Team90_Doe_Roe_Smith.py` and the report `Lab_<LabNumber>_Report_Team90_Doe_Roe_Smith.pdf`, where `<LabNumber>` is the number for that week's lab, e.g., **0, 1, 2**, etc. Submit your lab report along with any code to ELMS under the assignment for that week's lab **by the deadline stated on the ELMS assignment**. See the **Files** section on ELMS for a lab report template named `Lab_Report_Template.pdf`. The grading rubric is as follows:

Component	Points
Lab Report and Formatting	10
Question 1.	5
Question 2.	5
Question 3.	5
Question 4.	5
Question 5.	5
Question 6.	5
Question 7.	5
Question 8.	5
Exercise 1.	20
Exercise 2.	30
Total	100